applications. One key restriction is device security: No matter your role or the document you need, if you are an employee accessing from an unmanaged device, the request must be denied. This policy doesn't target a particular action or resource; instead, it functions as a broad rule layered over all others, ensuring every authorization decision aligns with ACME's zero-trust device posture:

```
forbid(
  principal is ACME::Employee,
  action,
  resource is ACME::Document
)
when { context.device.managed == false };
```

Note that this policy uses ALL for the action instead of listing them individually. This guarantees that the forbid applies to any new document actions ACME might add in the future, preventing gaps or drift if policies aren't updated alongside the schema.

We can encode this policy and upload it to the policy store as before:

```
jq -Rs --arg desc "Global forbid for unmanaged devices" \
   '{static: {description: $desc, statement: .}}' \
   policy-managed-device.cedar > policy-managed-device.json

aws verifiedpermissions create-policy \
  --policy-store-id $STORE_ID \
  --definition file://policy-managed-device.json
```

For this example, we uploaded this policy to the same policy store as the others. At ACME, the previous policies were uploaded to the policy store for the Customer Collaboration platform, managed by the team responsible for building it. The policy that forbids employee access on unmanaged devices was placed in a separate policy store managed by the security team.

A.5 Defining entities

Next, we'll upload entities to test policies and see how they work. In a real application, this data would come from a relationship graph stored in a